

Lab Manual

Name: E. Bharath

Reg No:192424421

19/06/2026

Experiment 31. Stacked Bar Plot with Error Bars

Aim

To create a stacked bar plot with error bars for men and women scores using Matplotlib.

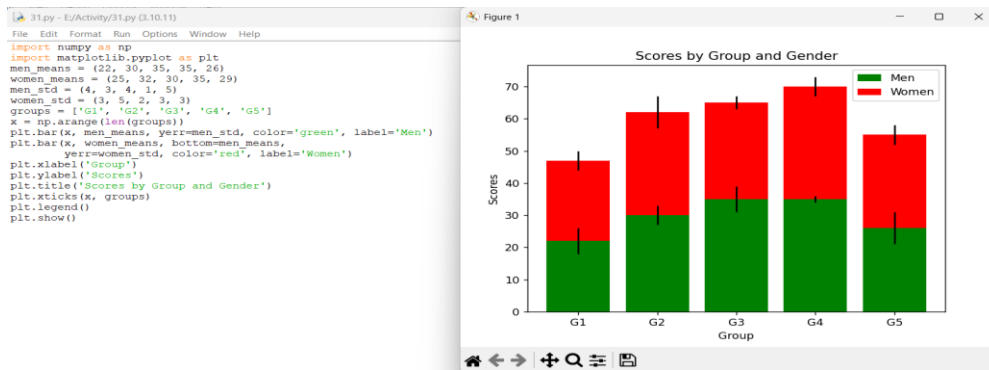
Algorithm

1. Import NumPy and Matplotlib.
2. Define men's and women's mean scores and standard deviations.
3. Create group labels. Plot men's bars with error bars.
4. Plot women's bars above men's bars using the bottom parameter.
5. Add labels, title, and legend. Display the chart.

Program

```
import numpy as np
import matplotlib.pyplot as plt
men_means = (22, 30, 35, 35, 26) women_means = (25, 32, 30, 35, 29)
men_std = (4, 3, 4, 1, 5) women_std = (3, 5, 2, 3, 3)
groups = ['G1', 'G2', 'G3', 'G4', 'G5']
x = np.arange(len(groups))
plt.bar(x, men_means, yerr=men_std, color='green', label='Men')
plt.bar(x, women_means, bottom=men_means,
        yerr=women_std, color='red', label='Women')
plt.xlabel('Group')
plt.ylabel('Scores')
plt.title('Scores by Group and Gender')
plt.xticks(x, groups)
plt.legend()
plt.show()
```

Output:



Result:

Thus, the stacked bar plot with error bars was created successfully.

19/06/2026

Experiment 32: Scatter Graph with Random Distribution

Aim

To write a Python program to generate random values for X and Y coordinates and display them using a scatter plot.

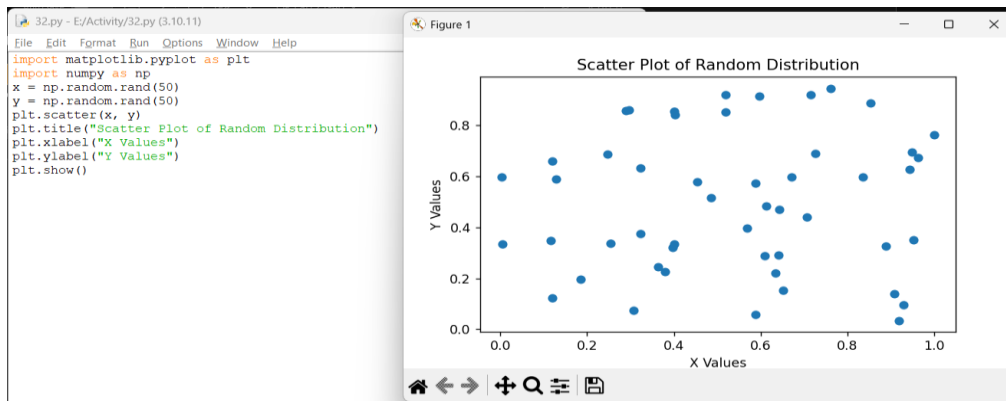
Algorithm

1. Start the program.
2. Import the required libraries: matplotlib.pyplot and numpy.
3. Generate random values for X coordinates and Y coordinates.
4. Use the scatter() function to plot X and Y values.
5. Add title and axis labels.
6. Display the scatter graph using show().
7. Stop the program.

Program

```
import matplotlib.pyplot as plt
import numpy as np
x = np.random.rand(50)
y = np.random.rand(50)
plt.scatter(x, y)
plt.title("Scatter Plot of Random Distribution")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.show()
```

Output:



Result:

Thus, the Python program to draw a scatter graph using random distributions in X and Y was successfully executed, and the scatter plot was displayed.

19/06/2026

Experiment No: 33 Scatter Plot with Empty Circles Taking a Random Distribution in X and Y

Aim

To write a Python program to generate random values for X and Y coordinates and display them as a scatter plot with empty circles.

Algorithm

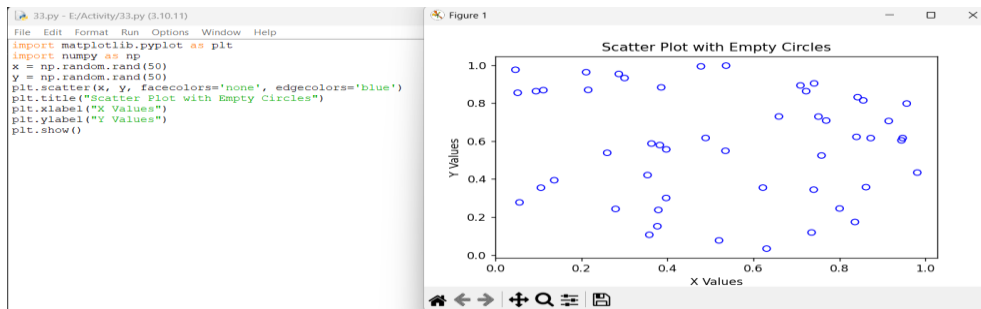
1. Start the program.
2. Import the required libraries matplotlib.pyplot and numpy.
3. Generate random values for X coordinates.
4. Generate random values for Y coordinates.
5. Use the scatter() function with facecolors='none' to create empty circles.
6. Specify edge color for the circles and Add title and axis labels.
7. Display the scatter plot using show().
8. Stop the program.

Program

```
import matplotlib.pyplot as plt
import numpy as np
x = np.random.rand(50)
y = np.random.rand(50)
plt.scatter(x, y, facecolors='none', edgecolors='blue')
plt.title("Scatter Plot with Empty Circles")
plt.xlabel("X Values")
```

```
plt.ylabel("Y Values")
plt.show()
```

Output:



Result:

Thus, the Python program to draw a **scatter plot with empty circles using random distributions in X and Y** was successfully executed and the required graph was displayed.

22/06/2026

Experiment No: 34 Scatter Plot Using Random Distributions to Generate Balls of Different Sizes

Aim

To write a Python program that generates random X and Y values and displays them as a scatter plot with markers (balls) of different sizes.

Algorithm

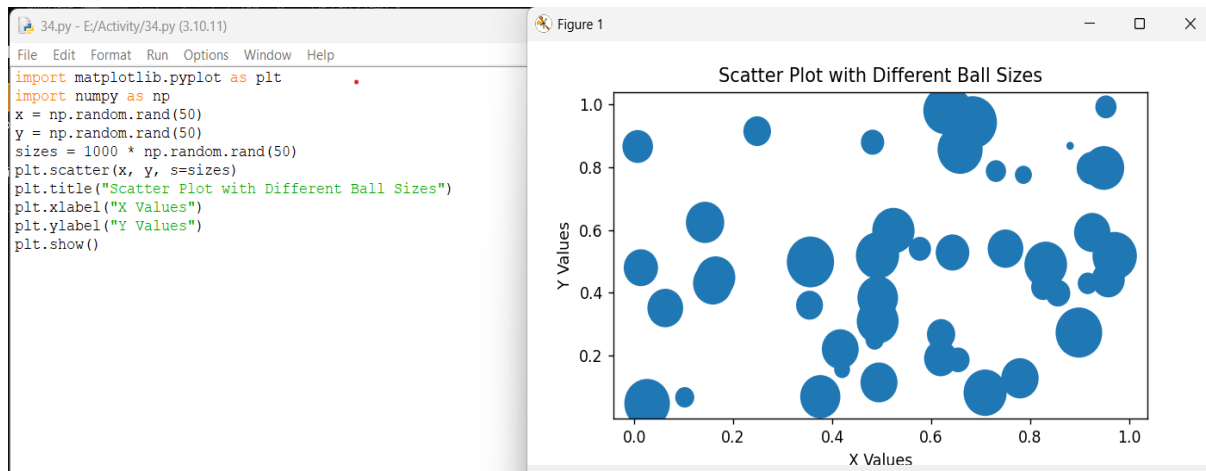
1. Start the program.
2. Import the required libraries matplotlib.pyplot and numpy.
3. Generate random values for X coordinates and Y coordinates.
4. Generate random values for marker sizes.
5. Use the scatter() function with the s parameter to specify different ball sizes.
6. Add title and axis labels. Display the scatter plot using show().
7. Stop the program.

Program

```
import matplotlib.pyplot as plt
import numpy as np
x = np.random.rand(50)
y = np.random.rand(50)
sizes = 1000 * np.random.rand(50)
plt.scatter(x, y, s=sizes)
plt.title("Scatter Plot with Different Ball Sizes")
plt.xlabel("X Values")
```

```
plt.ylabel("Y Values")
plt.show()
```

Output:



Result:

Thus, the Python program to draw a scatter plot using random distributions with balls of different sizes was successfully executed and the required graph was displayed.

22/06/2026

Experiment No: 35 Scatter Plot Comparing Mathematics and Science Marks of 10 Students

Aim

To write a Python program to draw a scatter plot comparing the marks obtained by 10 students in Mathematics and Science.

Algorithm

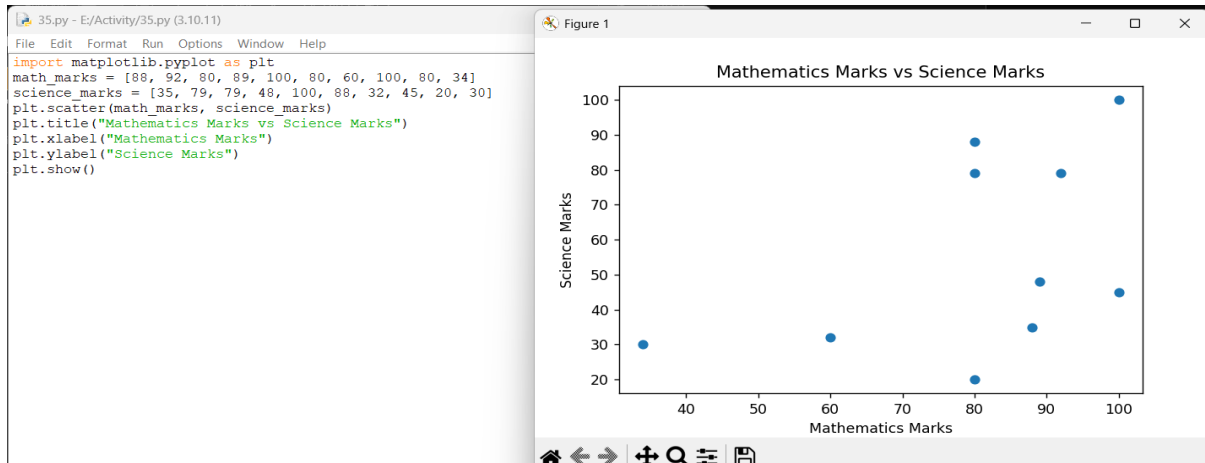
1. Start the program.
2. Import the matplotlib.pyplot library.
3. Create lists containing Mathematics marks and Science marks.
4. Use the scatter() function to plot Mathematics marks against Science marks.
5. Add title and labels for X-axis and Y-axis and display the scatter plot using show().
6. Stop the program.

Program

```
import matplotlib.pyplot as plt
math_marks = [88, 92, 80, 89, 100, 80, 60, 100, 80, 34]
science_marks = [35, 79, 79, 48, 100, 88, 32, 45, 20, 30]
plt.scatter(math_marks, science_marks)
plt.title("Mathematics Marks vs Science Marks")
```

```
plt.xlabel("Mathematics Marks")
plt.ylabel("Science Marks")
plt.show()
```

Output:



Result:

Thus, the Python program to draw a **scatter plot comparing Mathematics and Science marks of 10 students** was successfully executed and the required graph was displayed.

22/06/2026

Experiment No: 36 Scatter Plot for Three Different Groups

Aim

To draw a scatter plot for three different groups by comparing their weights and heights using Python and Matplotlib.

Algorithm

1. Import the required libraries (matplotlib.pyplot).
2. Define the height and weight data for three different groups.
3. Create a scatter plot for each group using different markers.
4. Label the X-axis as **Height (cm)** and Y-axis as **Weight (kg)**.
5. Add a title and legend to distinguish the groups.
6. Display the scatter plot.

Program

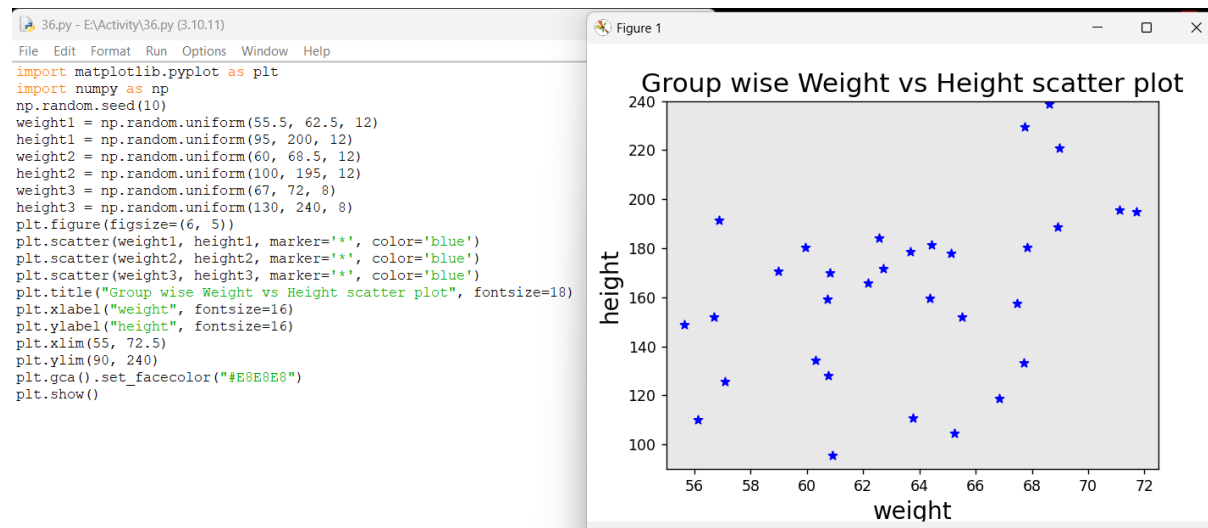
```
import matplotlib.pyplot as plt
import numpy as np
np.random.seed(10)
weight1 = np.random.uniform(55.5, 62.5, 12)
```

```

height1 = np.random.uniform(95, 200, 12)
weight2 = np.random.uniform(60, 68.5, 12)
height2 = np.random.uniform(100, 195, 12)
weight3 = np.random.uniform(67, 72, 8)
height3 = np.random.uniform(130, 240, 8)
plt.figure(figsize=(6, 5))
plt.scatter(weight1, height1, marker='*', color='blue')
plt.scatter(weight2, height2, marker='*', color='blue')
plt.scatter(weight3, height3, marker='*', color='blue')
plt.title("Group wise Weight vs Height scatter plot", fontsize=18)
plt.xlabel("weight", fontsize=16)
plt.ylabel("height", fontsize=16)
plt.xlim(55, 72.5)
plt.ylim(90, 240)
plt.gca().set_facecolor("#E8E8E8")
plt.show()

```

Output:



Result

Thus, the Python program to draw a scatter plot for three different groups comparing **weights** and **heights** was executed successfully, and the required scatter plot was generated.

23/06/2026

Experiment No: 37 Data frame from a Dictionary

Aim

To create a Pandas DataFrame from a dictionary and display it.

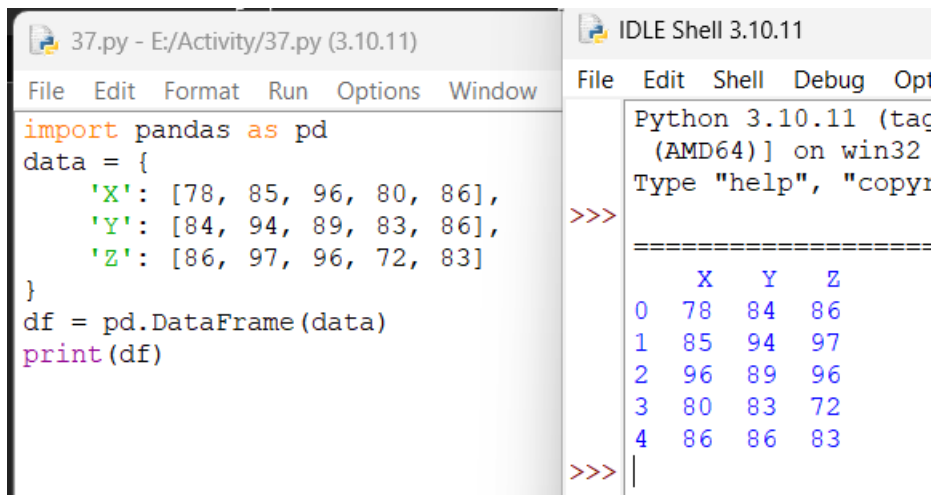
Algorithm

1. Import the Pandas library.
2. Create a dictionary containing sample data.
3. Convert the dictionary into a Pandas DataFrame.
4. Display the DataFrame.

Program

```
import pandas as pd
data = {
    'X': [78, 85, 96, 80, 86],
    'Y': [84, 94, 89, 83, 86],
    'Z': [86, 97, 96, 72, 83]
}
df = pd.DataFrame(data)
print(df)
```

Output

The screenshot shows a Python IDE with two windows. The left window, titled '37.py - E:/Activity/37.py (3.10.11)', contains the code to create a DataFrame from a dictionary. The right window, titled 'IDLE Shell 3.10.11', shows the execution output. The output displays the DataFrame with columns X, Y, and Z, and rows indexed from 0 to 4. The code in the left window is:

```
import pandas as pd
data = {
    'X': [78, 85, 96, 80, 86],
    'Y': [84, 94, 89, 83, 86],
    'Z': [86, 97, 96, 72, 83]
}
df = pd.DataFrame(data)
print(df)
```

The output in the right window is:

```
>>>
=====
      X   Y   Z
0  78  84  86
1  85  94  97
2  96  89  96
3  80  83  72
4  86  86  83
>>>
```

Result

Thus, the Pandas program to create a DataFrame from a dictionary and display it was executed successfully.

23/06/2026

Experiment No: 38 Specified Dictionary Data

Aim

To create and display a Pandas DataFrame from a specified dictionary using custom index labels.

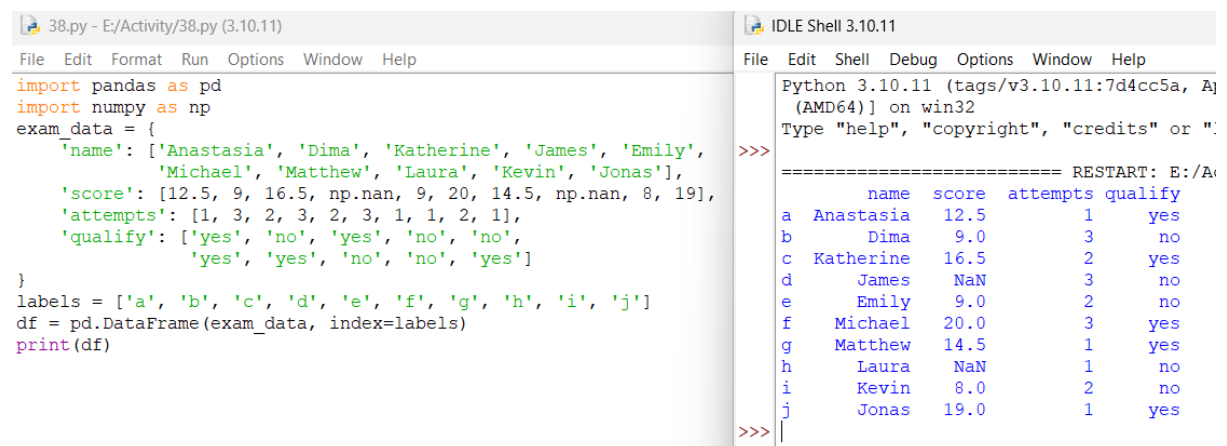
Algorithm

1. Import the Pandas and NumPy libraries.
2. Create a dictionary containing the given data.
3. Create a list of index labels.
4. Convert the dictionary into a Pandas DataFrame using the specified index labels.
5. Display the DataFrame.

Program

```
import pandas as pd
import numpy as np
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
              'yes', 'yes', 'no', 'no', 'yes']
}
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
df = pd.DataFrame(exam_data, index=labels)
print(df)
```

Output



The screenshot shows a Python IDE with two windows. The left window displays the code from the 'Program' section. The right window, titled 'IDLE Shell 3.10.11', shows the execution output. It starts with a 'RESTART: E:/A/' prompt, followed by a table representation of the DataFrame. The table has columns 'name', 'score', 'attempts', and 'qualify', indexed by letters 'a' through 'j'. The output matches the data defined in the code.

	name	score	attempts	qualify
a	Anastasia	12.5	1	yes
b	Dima	9.0	3	no
c	Katherine	16.5	2	yes
d	James	NaN	3	no
e	Emily	9.0	2	no
f	Michael	20.0	3	yes
g	Matthew	14.5	1	yes
h	Laura	NaN	1	no
i	Kevin	8.0	2	no
j	Jonas	19.0	1	yes

Result

Thus, the Pandas program to create and display a DataFrame from a specified dictionary using custom index labels was executed successfully.

23/06/2026

Aim

To get the first three rows of a given Pandas DataFrame.

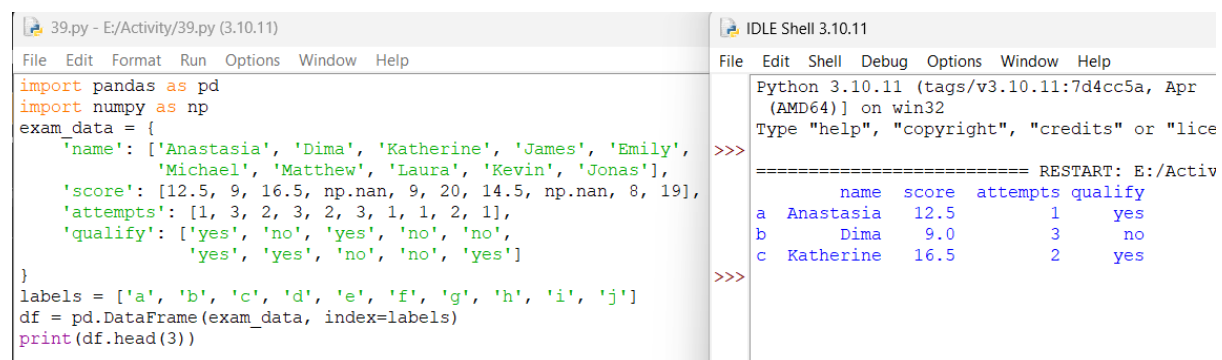
Algorithm

1. Import the Pandas and NumPy libraries.
2. Create a dictionary containing the given data.
3. Create a list of index labels.
4. Create a DataFrame using the dictionary and index labels.
5. Use the head(3) method to display the first three rows.
6. Display the result.

Program

```
import pandas as pd
import numpy as np
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
              'yes', 'yes', 'no', 'no', 'yes']
}
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
df = pd.DataFrame(exam_data, index=labels)
print(df.head(3))
```

Output



The screenshot shows a Python IDE with two panes. The left pane displays the code from the 'Program' section. The right pane shows the output of the code, which is a table of the first three rows of the DataFrame.

```
39.py - E:/Activity/39.py (3.10.11)
File Edit Format Run Options Window Help
import pandas as pd
import numpy as np
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
              'yes', 'yes', 'no', 'no', 'yes']
}
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
df = pd.DataFrame(exam_data, index=labels)
print(df.head(3))

IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr
(AMD64)) on win32
Type "help", "copyright", "credits" or "lice
===== RESTART: E:/Activ
   name  score  attempts  qualify
a  Anastasia   12.5         1     yes
b    Dima      9.0         3     no
c  Katherine   16.5         2     yes
```

Result

Thus, the Pandas program to retrieve and display the first three rows of the given DataFrame was executed successfully.

23/06/2026

Experiment No: 40 Select Name and Score

Aim

To select the '**name**' and '**score**' columns from a given Pandas DataFrame.

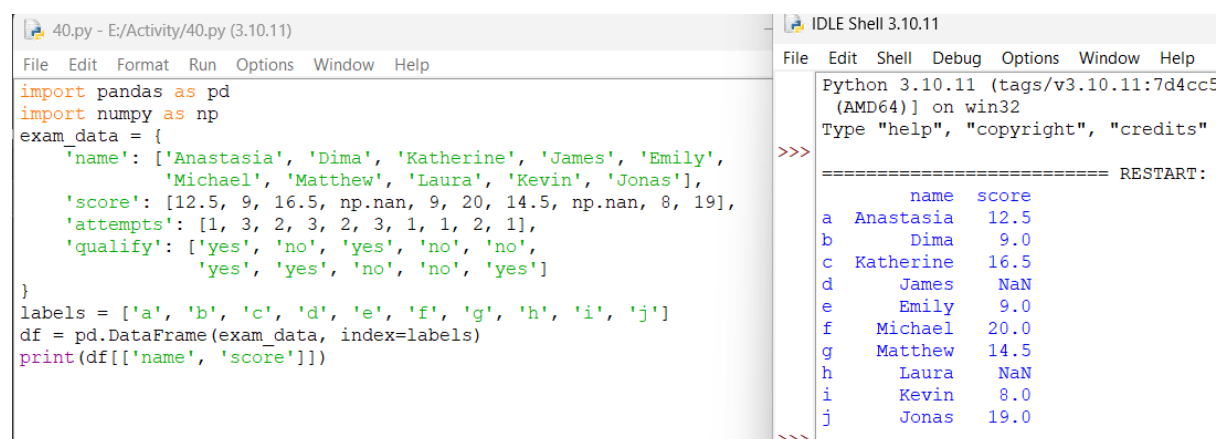
Algorithm

1. Import the Pandas and NumPy libraries.
2. Create a dictionary containing the given data.
3. Create a list of index labels.
4. Create a DataFrame using the dictionary and index labels.
5. Select the '**name**' and '**score**' columns using column indexing.
6. Display the selected columns.

Program

```
import pandas as pd
import numpy as np
exam_data = {
    'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily',
            'Michael', 'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no',
              'yes', 'yes', 'no', 'no', 'yes']
}
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
df = pd.DataFrame(exam_data, index=labels)
print(df[['name', 'score']])
```

Output



The screenshot shows a Python IDE with a file named '40.py' and an IDLE Shell. The code in the file defines a dictionary 'exam_data' with columns 'name', 'score', 'attempts', and 'qualify'. It then creates a list 'labels' and a DataFrame 'df' using 'exam_data' and 'labels' as the index. Finally, it prints the DataFrame columns 'name' and 'score'. The shell output shows the DataFrame with the following data:

	name	score
a	Anastasia	12.5
b	Dima	9.0
c	Katherine	16.5
d	James	NaN
e	Emily	9.0
f	Michael	20.0
g	Matthew	14.5
h	Laura	NaN
i	Kevin	8.0
j	Jonas	19.0

Result

Thus, the Pandas program to select and display the 'name' and 'score' columns from the given DataFrame was executed successfully.